Tourism AI System

What is implemented, what was tested, and what still needs attention

ROUTING: VERIFIED
Mapbox Directions request and a full LangGraph run succeeded. The test produced a feasible 2-stop itinerary with 6 verified route estimates.

AGENTS: EXECUTING
Manager, Research, Route, Verification, Itinerary, and Response nodes ran. Structured traces recorded all six nodes.

GEMINI: NOT VERIFIED
The Gemini request timed out from this environment. The app falls back to deterministic intent parsing when Gemini cannot answer.

DATA: LOCAL JSON
12 curated Gwalior POIs load from a local JSON file. MongoDB is not wired into the application.

How a trip request flows

1. Browser	2. Express / Node	3. LangGraph / Flask
Traveler selects interests, date, times, transport, and return point.	Receives POST /api/v1/tourism-plan and forwards the request to Flask.	Runs six workflow nodes and returns a structured itinerary.
4. Research	**5. Mapbox + Verification**	**6. Itinerary + Response**
Reads exact matches from the local Gwalior JSON dataset.	Gets directed road estimates; labels route source and confidence.	Checks timing and return deadline, then sends results to the UI.

Testing completed

Suite	Result	Notes
Python / LangGraph	**33 passed**	Includes real graph execution, Mapbox adapter/unit tests, failure branches, and demo-path tests. One provider integration test is excluded from this suite.
Node / Express	**9 passed**	Includes page/API route checks and forwarding contract. Dataset test is isolated from live route calls.
Live Mapbox	**Passed**	One real route request and one end-to-end plan run succeeded with Mapbox route sources classified VERIFIED.
Live Gemini	**Timed out**	Bounded 15-second calls did not receive a response in this environment. The supplied key has not been validated here.

What remains

The important difference is between the code configuration and the processes that are already running.

Restart the services

The existing Flask process still runs with its earlier demo-mode environment. Restart Flask so requests from Express use the Mapbox configuration now stored in the project .env. Restart Express as well so it reloads the Gemini model setting and key. Then submit one trip through the browser and confirm the response lists mapbox_driving routes, not synthetic_demo_fixture.

Keys and configuration

Setting	Purpose / current state
GEMINI_API_KEY	Gemini intent parsing. Stored locally; validity is unverified because calls timed out.
GEMINI_MODEL=gemini-flash-latest	Model alias shared by the user; app reads GEMINI_MODEL or defaults to this alias.
ROUTING_PROVIDER_URL	Set to https://api.mapbox.com. Configured and tested.
ROUTING_PROVIDER_API_KEY	Mapbox access token. Stored locally; never print or send it to the browser.
DEMO_MODE=false	Required in the Flask process environment for real provider calls.
MONGODB_URI	Not needed currently. No MongoDB adapter is wired; the planner reads local JSON.

Known limitations

• Gemini has not been confirmed working from this environment. Until a live response succeeds, intent parsing falls back to deterministic rules.

• Mapbox driving profile is a road-route estimate; it is not tuned for auto-rickshaw, e-rickshaw, or cab-specific speeds. Public transit is not routed by this adapter.

• The curated dataset currently covers Gwalior only. A Jaipur request correctly returns no matching places.

• The app uses a local JSON file, not MongoDB. Do not describe MongoDB as connected.

• The supplied Gemini and Mapbox credentials were pasted into chat. Rotate them for security, then update the local .env.

Hackathon demo checklist

1	Restart Flask and Express; confirm DEMO_MODE=false in the actual Flask process.
2	Show one successful trip and the six-node execution trace.
3	Show route source labels and the return-by-deadline check.
4	Show missing-input and no-match behavior; explain local JSON and Gemini fallback honestly.

Prepared from the project audit and tests run on 29-30 Oct 2026. This is a progress snapshot, not a promise of a hackathon result.